

Bin Robot

Operating manual — mapping, navigation and waypoint routes

Workspace [bin_ws](#) · [ROS 2 Jazzy](#) · [Ubuntu, CycloneDDS](#)

Packages [bin_bringup](#) · [bin_description](#) · [bin_hardware](#) · [bin_behavior](#) · [bin_perception](#)

Firmware [ROSArduinoBridge_BTS7960](#) (Arduino Nano, ATmega328P)

1 What runs where

The system spans two machines. Getting this wrong is the single most common source of confusion, because **each machine has its own checkout of the workspace**.

Machine	Runs	Responsible for
ROBOT reached over ssh	<code>bin_real.launch.py</code>	Motors and encoders over <code>/dev/bin_motors</code> , LD19 laser over <code>/dev/bin_lidar</code> , <code>robot_state_publisher</code> , <code>ros2_control</code> , the scan gate.
LAPTOP	<code>laptop_slam_nav2.launch.py</code> <code>laptop_nav_waypoints.launch.py</code>	SLAM or AMCL localisation, Nav2, RViz, the waypoint tools.

Edits do not cross the network

Changing a file on the laptop does nothing to the robot until you sync the workspace and run `colcon build --symlink-install` *on the robot*. If a fix appears to have no effect, check the robot's copy of the file before assuming the change was wrong.

Topic flow

```
# command path
Nav2 controller → cmd_vel_nav → velocity_smoother → cmd_vel_smoothed
                → collision_monitor → /cmd_vel → twist_stamper
                → /diff_drive_controller/cmd_vel → wheels

# sensing path
LD19   → /lidar_raw → scan_gate → /lidar → slam_toolbox / AMCL / costmaps
camera → /camera/rgb/image_raw{,/compressed} + /camera/rgb/camera_info
                                             → color_detector / yolo_detector
encoders → ros2_control → /joint_states + /diff_drive_controller/odom
                → odom → base_footprint (TF)
```

2 Hardware reference

Arduino Nano pin map

Every PWM pin on the Nano is spoken for. This table is why the lid servo is bit-banged rather than driven by `<Servo.h>` — see section 10.

Pin	Function	Timer	Notes
D2	Left encoder A	—	PCINT2 / PORTD
D3	Left encoder B	Timer2	PWM unused — digital input
D5	Left motor LPWM	Timer0	Timer0 also drives millis()
D6	Right motor LPWM	Timer0	
D9	Left motor RPWM	Timer1	Why Servo.h is unusable here
D10	Right motor RPWM	Timer1	
D11	Lid servo	Timer2	Bit-banged, no timer used
A4 / A5	Right encoder A / B	—	PCINT1 / PORTC

Key constants

Parameter	Value	Where
Motor board serial	/dev/bin_motors @ 57600	bin_real.launch.py motor_port
Laser serial	/dev/bin_lidar @ 230400	bin_real.launch.py lidar_port
Lid hold-open window	1500 ms	servo_driver.h
Camera	CSI, 640x480	camera_ros via libcamera
Controller update rate	20 Hz	bin_controllers.yaml
Serial read timeout	50 ms	bin_ros2_control.xacro
Encoder counts / rev	2475	bin_ros2_control.xacro
Wheel radius	0.031755 m	bin_controllers.yaml
Wheel separation	0.2484 m	bin_controllers.yaml
Robot radius (costmap)	0.15 m	nav2_params.yaml
Max speed (linear / angular)	0.15 m/s / 0.6 rad/s	nav2_params.yaml

3 One-time setup

Both machines

```
# Pick any number, but it must match on both machines. Without this every ROS 2
# process on the WiFi discovers every other one, and other robots on the LAN reuse
# the same map/odom/base_footprint frame names.
echo 'export ROS_DOMAIN_ID=17' >> ~/.bashrc
echo 'export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp' >> ~/.bashrc
source ~/.bashrc
```

Build

```
cd ~/bin_ws
colcon build --symlink-install
source install/setup.bash
```

`--symlink-install` matters: it makes config and Python files live, so editing a yaml or a node takes effect on the next run without rebuilding. C++ changes (`bin_hardware`) and new launch files still need a rebuild.

Sync to the robot

```
rsync -av --delete ~/bin_ws/src/ <user>@<robot>.local:~/bin_ws/src/
ssh <user>@<robot>.local
cd ~/bin_ws && colcon build --symlink-install && source install/setup.bash
```

Camera (camera_ros)

The CSI camera is driven by `camera_ros` , built from source in `src/camera_ros` rather than installed from apt. The released `ros-jazzy-camera-ros` depends on `ros-jazzy-libcamera` , ROS's own vendored build, which generally lacks the `rpi/vc4` pipeline handler and so never enumerates a Pi camera. Building here links the system libcamera instead.

Check libcamera can see the sensor before anything else — if it cannot, neither can ROS and the problem is below ROS:

```
rpicam-hello --list-cameras      # or: cam -l
```

If it is missing, check `camera_auto_detect=1` and the right `dtoverlay` in `/boot/firmware/config.txt` , then reboot.

```
--- ROBOT ---
cd ~/bin_ws
rosdep install -y --from-paths src --ignore-src --rosdistro jazzy --skip-keys=libcamera
colcon build --symlink-install --packages-select camera_ros
```

`--skip-keys=libcamera` is what stops rosdep pulling the ROS-vendored libcamera back in over the system one.

The laptop needs libcamera-dev, or camera_ros skipped

camera_ros lives in src/, so a plain colcon build on the laptop tries to build it and fails with libcamera>=0.1 not found. Either sudo apt install libcamera-dev once (0.2.0 satisfies it, and everything then just builds), or pass --packages-ignore camera_ros on the laptop. Do

not

drop a COLCON_IGNORE in src/camera_ros — rsync carries it to the robot and skips the package exactly where it is needed.

Stable device names — required

/dev/ttyUSB0 and /dev/ttyUSB1 are handed out in enumeration order. If the Arduino resets and re-enumerates, the two devices swap and the motor driver starts speaking its protocol at the laser. Give them fixed names once:

```
--- ROBOT, both devices plugged in ---
cd ~/bin_ws/src/bin/bin_bringup/udev
./setup_udev.sh                # inspect devices, print suggested rules
```

The script prints a rule per device with the symlink name left as bin_CHANGEME. Work out which is which — with the ROS stack stopped, the motor board is the one that answers the encoder query:

```
printf 'e\r' > /dev/ttyUSB0 ; timeout 1 cat /dev/ttyUSB0
# two integers back = that is the Arduino; the other is the LD19
```

Then edit the names to bin_motors and bin_lidar, save the block to /etc/udev/rules.d/99-bin-robot.rules, and reload:

```
sudo udevadm control --reload-rules && sudo udevadm trigger
ls -l /dev/bin_motors /dev/bin_lidar
```

CH340 chips have no serial number

If both adapters are CH340s they are indistinguishable by ID, so the script falls back to matching the physical USB port. That works, but the devices must then stay in the same hub sockets.

To run without the rules, pass the raw ports:

```
ros2 launch bin_bringup bin_real.launch.py \
  motor_port:=/dev/ttyUSB0 lidar_port:=/dev/ttyUSB1
```

Firmware

Flash ROSArduinoBridge_BTS7960.ino to the Nano from the Arduino IDE. Before flashing, confirm DEBUG_PID_SERIAL is commented out in diff_controller.h — it prints onto the same serial port ROS reads, and its lines land in the middle of encoder replies.

4 Building a map

Do this once per site, then reuse the saved map for every navigation run.

```
--- ROBOT ---
ros2 launch bin_bringup bin_real.launch.py

--- LAPTOP ---
ros2 launch bin_bringup laptop_slam_nav2.launch.py
ros2 run teleop_twist_keyboard teleop_twist_keyboard
```

Drive the whole area. Good practice:

- Move slowly — the laser is 10 Hz and fast turns smear the scan match.
- Prefer long straight runs along walls; revisit the start point to close the loop.
- **Keep the lid closed the entire time.** The laser is mounted on it (section 10).
- Watch RViz: if the map tears or jumps, stop, back up and re-approach slowly.

Capture waypoints on the same drive

slam_toolbox publishes `map` → `odom` while mapping, so the capture tool in section 7 works during this run. Recording bins and the dock as you map saves a second lap.

Saving the map

```
--- LAPTOP, with the mapping run still going ---
ros2 run nav2_map_server map_saver_cli -f bin_floor

cp bin_floor.yaml bin_floor.pgm ~/bin_ws/src/bin/bin_bringup/maps/
cd ~/bin_ws && colcon build --symlink-install --packages-select bin_bringup
```

`bin_floor` is the name the navigation launch file expects by default. Use a different one with `map:=/full/path/to/other.yaml`.

5 Navigating a saved map

```
--- ROBOT ---
ros2 launch bin_bringup bin_real.launch.py

--- LAPTOP ---
ros2 launch bin_bringup laptop_nav_waypoints.launch.py
# or with a different map:
ros2 launch bin_bringup laptop_nav_waypoints.launch.py map:=/path/to/other.yaml
```

Seeding the position

AMCL starts believing the robot is at the map origin (`set_initial_pose: true` in `nav2_params.yaml`). If the robot starts anywhere else, correct it: in RViz click **2D Pose Estimate**, then click-and-drag on the robot's real position pointing the way it faces.

Confirm it took:

```
ros2 topic echo /amcl_pose
ros2 run tf2_ros tf2_echo map base_footprint
```

Drive a few metres. A healthy AMCL shows the covariance shrinking and the particle cloud in RViz tightening around the robot. A cloud that stays spread out, or snaps to a wrong place, means the initial pose was too far off — re-seed it.

6 Driving manually

```
ros2 run teleop_twist_keyboard teleop_twist_keyboard
```

Publishes `/cmd_vel`, which `twist_stamper` converts and forwards to the controller. Works in every mode, including while Nav2 is running — useful for nudging the robot out of a corner, though Nav2 will fight you if it has an active goal.

Check	Expected
<code>ros2 topic hz /diff_drive_controller/odom</code>	~20 Hz
<code>ros2 topic hz /lidar</code>	~10 Hz, lid closed
<code>ros2 topic echo /joint_states</code>	wheel positions changing as you drive
<code>ros2 run tf2_ros tf2_echo odom base_footprint</code>	advances smoothly, never jumps backward

7 Recording waypoints

Waypoints are recorded by **driving the robot to each spot and capturing where it actually is**. That guarantees a reachable pose with a real heading, rather than a clicked point that might sit inside a wall's inflation zone.

```
--- LAPTOP, with a mapping or navigation run already going ---
ros2 run bin_behavior waypoint_capture
ros2 run teleop_twist_keyboard teleop_twist_keyboard
```

Then for each stop: drive there, point the robot the way you want it to end up, and

```
# record this spot, and cycle the lid when the route reaches it
ros2 service call /waypoint_capture/capture std_srvs/srv/SetBool "{data: true}"

# record a drive-through corner instead – no lid
ros2 service call /waypoint_capture/capture std_srvs/srv/SetBool "{data: false}"

# mistakes
ros2 service call /waypoint_capture/undo std_srvs/srv/Trigger
ros2 service call /waypoint_capture/clear std_srvs/srv/Trigger
```

Each call replies with what it stored, so the terminal doubles as a log of the route:

```
success=True, message='waypoint 1: x=1.250 y=-0.750 yaw=1.571 open_lid=True'
```

The file is written after every call — there is no separate save step, and a crash never loses more than nothing. On a new site, call `clear` first to drop the shipped example route.

"No map → base_footprint transform"

The capture failed because nothing is localising. Start the SLAM or navigation launch first, and if you are in navigation mode, give AMCL its initial pose in RViz.

8 waypoints.yaml

Lives at `bin_bringup/config/waypoints.yaml`. Written by the capture tool, and safe to hand-edit.

```
waypoints:
- {x: 1.25, y: -0.75, yaw: 1.571, open_lid: true}
- {x: 3.5, y: 2.0, yaw: -0.785, open_lid: false}
loop: true
lid_dwell: 4.5
```

Key	Type	Meaning
x, y	metres	Position in the <code>map</code> frame.
yaw	radians	Heading, counter-clockwise from +x. 0 faces map +x, 1.571 faces +y.
open_lid	bool	Stop here and cycle the lid. Omit for a drive-through point.
loop	bool	Repeat the route until the runner is stopped.
lid_dwell	seconds	How long the lid stays open at each stop.

9 Running a route

From the file

```
ros2 run bin_behavior waypoint_runner
# or start it with the navigation stack:
ros2 launch bin_bringup laptop_nav_waypoints.launch.py run_waypoints:=true
```

The runner hands the route to Nav2's `waypoint_follower`, splitting it into segments at each `open_lid` stop. At those stops it opens the lid, waits `lid_dwell` seconds, closes it, waits for the laser to come back, and continues. It aborts the whole route if a segment fails, so a robot that cannot reach a bin stops rather than silently skipping it.

Ad-hoc, from RViz

For a one-off run with no file: in the Nav2 panel switch to **Waypoint / Nav Through Poses** mode, place goals with the **Nav2 Goal** tool, then press **Start Navigation**. Nothing is saved, and the lid is not cycled.

10 The lid and the laser

The LD19 is bolted to the lid, not the body. In the model that is `bin_top_link` → `bin_cover_link` (the revolute `bin_cover_joint`) → `lidar_link`. Opening the lid pitches the scanner by up to 90°, so it sweeps a vertical plane — floor and ceiling — instead of the room.

Left unhandled that would corrupt the map, mark phantom obstacles in the costmaps and raytrace-clear real ones. So `scan_gate` sits between the driver and everything else: the LD19 publishes `/lidar_raw`, and `/lidar` only carries scans taken while the lid was down and settled.

You will see this in the log, and it is normal:

```
[scan_gate]: lid not closed – blocking scans
[scan_gate]: lid closed and settled – passing scans
```

Condition	Behaviour
Lid closed, settled 0.5 s	Scans pass.
Lid commanded open	Scans stop immediately, before the servo has moved.
Lid closing	Scans stay blocked for 0.5 s after it reports closed, covering the servo's ~300 ms sweep.
<code>/joint_states</code> stops	Fails closed — no scans. A silent controller is treated as unknown lid state.

Why the lid does not stay held open

A bit-banged servo only holds position while it is being pulsed, and a servo holding against the lid's weight draws current the whole time — far more if it is pressed against a mechanical end stop. On a Nano powered over USB that sag is enough to brown out the board, which then resets and re-enumerates on a different `/dev/ttyUSB` node, killing the serial link mid-run.

So the firmware energises the servo for the ~300 ms sweep plus `SERVO_HOLD_OPEN_MS` (1500 ms), then stops pulsing and lets it relax. If the lid falls sooner than you want, the fix is electrical, not a longer window: give the servo its own 5 V supply with a common ground and a 470–1000 µF capacitor across it near the servo, then set `SERVO_HOLD_OPEN_MS` to 0 to hold indefinitely.

Only open the lid while stopped

With the laser gated the robot has no obstacle sensor. Nav2's `collision_monitor` latches a stop once its 1 s source timeout expires, which is the safety net, not the plan. `behavior_manager` and `waypoint_runner` both stop before opening.

10b Camera

Starts with `bin_real.launch.py` by default. Turn it off when you need the Pi's CPU or the WiFi for something else:

```
ros2 launch bin_bringup bin_real.launch.py camera:=false
```

Topic	Type
/camera/rgb/image_raw	sensor_msgs/Image
/camera/rgb/image_raw/compressed	sensor_msgs/CompressedImage
/camera/rgb/camera_info	sensor_msgs/CameraInfo

These are the same names the sim's gz bridge publishes, and the frame is `camera_depth_frame` — the optical frame, matching the sim's `gz_frame_id`. So `color_detector` and `yolo_detector` run unchanged against either.

Never subscribe to the raw image across WiFi
640×480 RGB at 30 fps is about 27 MB/s. It will swamp the link and starve the laser and TF traffic, which shows up as navigation falling apart rather than as an obvious camera problem. On the laptop always use `/camera/rgb/image_raw/compressed`. `jpeg_quality` is set to 75; the `FrameDurationLimits` parameter caps the frame rate if you need more.

11 Serial protocol

The Nano speaks a single-letter line protocol at 57600 baud, terminated by carriage return. Useful for bench-testing motors and the lid with no ROS running — close the ROS stack first, as only one process can hold `/dev/ttyUSB0`.

```
screen /dev/ttyUSB0 57600      # or the Arduino IDE serial monitor, line ending = CR
```

Cmd	Arguments	Does
e	—	Read encoders. Replies <code><left> <right></code> .
m	left right	Set speed in encoder counts per PID frame (closed loop).
o	left right	Raw PWM, <code>-255...255</code> , bypasses the PID.
r	—	Reset encoder counts to zero.
s	1 or 0	Lid open / closed.
q	—	Diagnostics: encoders, target and actual RPM, error, integral, PWM, velocities.
u	<code>lKp:lKd:lKi:lKo rKp:rKd:rKi:rKo</code>	Set both PIDs.
l	<code>Kp:Ki:Kd:MinPwm</code>	Left PID only. Note the different argument order.
f	<code>Kp:Ki:Kd:MinPwm</code>	Right PID only.
b	—	Report baud rate.
p	pin	Ping sensor read.

Bench test that catches most problems

```
e          # note the numbers
o 120 120  # both wheels forward
e          # both counts must have increased
o 0 0 0
s 1        # lid opens, replies OK immediately
s 0
```

If `s 1` takes a noticeable moment to reply, you are running old firmware — the lid used to block for 300 ms and stall everything else with it.

12 Troubleshooting

Symptom	Cause	Fix
Robot model jumps between two poses, one lagging ("the ghost")	Encoder replies being dropped or corrupted, so the wheel position momentarily reads zero and odometry teleports.	Confirm <code>DEBUG_PID_SERIAL</code> is commented out in <code>diff_controller.h</code> and reflash. Watch for <code>encoder read failed</code> warnings.
<code>encoder read failed; holding last position</code> repeating	The 57600 link is dropping replies.	Occasional is harmless. Constant means a bad USB cable, or <code>update_rate</code> raised too high for the baud rate.
Model at the origin, RViz shows TF errors	No <code>odom</code> → <code>base_footprint</code> .	<code>enable_odom_tf</code> must be true in <code>bin_controllers.yaml</code> . Check <code>tf2_echo odom base_footprint</code> .
Nothing appears with Fixed Frame <code>map</code>	No <code>map</code> → <code>odom</code> : SLAM or AMCL is not running or has no laser.	Check <code>ros2 topic hz /lidar</code> . If <code>/lidar_raw</code> flows but <code>/lidar</code> does not, the lid is open — see section 10.
controller_manager aborts: <i>Joint 'bin_cover_joint' is of type 'fixed'</i>	The robot's <code>bin_description</code> is out of sync with the laptop's.	Re-sync the whole <code>src/</code> tree and rebuild on the robot. Never copy individual files.
Controllers never activate; spawners retry forever	<code>ros2_control_node</code> died at startup — look above the retries for the real error.	Usually a URDF mismatch or <code>/dev/ttyUSB0</code> busy or absent.
Everything runs but nothing moves	Nav2 running on simulated time with no clock.	<code>use_sim_time</code> must be <code>False</code> throughout <code>nav2_params.yaml</code> . Verify with <code>ros2 param get /controller_server use_sim_time</code> .
Robot drifts off the map, particle cloud spread wide	AMCL lost, or seeded badly.	Re-seed with 2D Pose Estimate. If it keeps happening the map is poor — re-map more slowly.
Serial dies after a lid command; device reappears as a different ttyUSB number	Servo current browning out the Nano, which resets and re-enumerates.	Shorten <code>SERVO_HOLD_OPEN_MS</code> , or better, move the servo to its own 5 V supply. Install the udev rules so a reset cannot swap the ports.
<i>device reports readiness to read but returned no data</i>	The port vanished underneath an open connection — same re-enumeration, or two processes on one port.	Check <code>dmesg tail</code> for USB disconnect lines. Confirm the ROS stack is not already holding the port.
No camera topics; <code>camera_ros</code> logs no cameras found	<code>libcamera</code> cannot see the sensor, or it was built against the ROS-vendored <code>libcamera</code> .	Run <code>rplicam-hello --list-cameras</code> first. If that works but ROS does not, rebuild with <code>--skip-keys=libcamera</code> .
colcon build fails: <code>libcamera>=0.1</code> not found	Building <code>camera_ros</code> on a machine without <code>libcamera</code> headers — normally the laptop.	<code>sudo apt install libcamera-dev</code> , or build with <code>--packages-ignore camera_ros</code> .
Navigation degrades once the camera is on	Raw image saturating the WiFi and starving laser and TF traffic.	Subscribe to the compressed topic only, or run with <code>camera:=false</code> .

Laptop sees no topics from the robot	Mismatched <code>ROS_DOMAIN_ID</code> , or different WiFi.	<code>echo \$ROS_DOMAIN_ID</code> on both. Both must be identical and on the same subnet.
Route skips a bin, or stops early	A segment failed; the runner aborts rather than continuing.	Read the runner's log for which waypoint. Usually unreachable — recapture it slightly further from the wall.

13 Cheat sheet

ROBOT

```
ssh <user>@<robot>.local
cd ~/bin_ws && source install/setup.bash

# always the first thing to start
ros2 launch bin_bringup bin_real.launch.py

# after any code or config change
colcon build --symlink-install
```

Map building

```
ros2 launch bin_bringup \
  laptop_slam_nav2.launch.py

ros2 run nav2_map_server \
  map_saver_cli -f bin_floor
```

LAPTOP

```
cd ~/bin_ws && source install/setup.bash

# navigation on the saved map
ros2 launch bin_bringup \
  laptop_nav_waypoints.launch.py

# drive by hand
ros2 run teleop_twist_keyboard \
  teleop_twist_keyboard
```

Waypoints

```
ros2 run bin_behavior waypoint_capture
ros2 service call \
  /waypoint_capture/capture \
  std_srvs/srv/SetBool "{data: true}"

ros2 run bin_behavior waypoint_runner
```

Health checks

```
ros2 topic hz /lidar # ~10 Hz, lid closed
ros2 topic hz /diff_drive_controller/odom # ~20 Hz
ros2 run tf2_ros tf2_echo odom base_footprint # smooth, no backward jumps
ros2 run tf2_ros tf2_echo map odom # exists once SLAM/AMCL is up
ros2 topic echo /amcl_pose # covariance shrinking
ros2 node list # one of each, no duplicates
```